

ShipShape Phase 2 Final Audit Report

Date: 2026-03-16 **Application:** ShipShape (project management application) **Scope:** `api/src/` , `web/src/` , `shared/src/` , `e2e/` **Baseline Branch:** `master` (pre-optimization) **Database:** Seeded with 501 documents, 218 issues, 22 users, 35 sprints

1. Executive Summary

This report documents the results of a structured optimization effort across 7 categories of the ShipShape codebase. Each category was audited against the original `master` baseline, assigned specific improvement targets, and implemented on an isolated branch with measurable results.

Key results:

Metric	Value
Categories audited and improved	7 of 7
Improvement targets met or exceeded	7 of 7
Specs implemented (Phase 2)	24
Additional test fixes (Phase 3)	30+ commits
Tests restored to operational	+1,012
Type violation reduction (explicit)	-44%
API response time improvement (wiki p99)	-50%
Database query reduction (main page)	-36%
Critical/Serious WCAG violations eliminated	13 of 13
Silent runtime failures fixed	5 of 6

All improvements are tied to specific commits with structured messages documenting the problem, fix, tradeoffs, and measured improvement. Every category branch was merged to `master` with `--no-ff` to preserve branch history.

2. Methodology

Baseline Measurement

All baseline measurements were taken from the original `master` branch against a seeded PostgreSQL database containing production-representative data (501 documents, 218 issues, 22 users, 35 sprints). The original audit files are archived in `docs/benchmark/phase1-baseline_Audit/01-*.md` through `docs/benchmark/phase1-baseline_Audit/07-*.md`.

Branching Strategy

Each category was implemented on its own branch, merged sequentially to `master`:

Category	Branch	DB Changes
Cat 1 -- Type Safety	<code>cat1-type-safety</code>	No
Cat 2 -- Bundle Size	<code>cat2-bundle-size</code>	No
Cat 3 -- API Response Time	<code>cat3-api-response-time</code>	No
Cat 4 -- DB Query Efficiency	<code>cat4-db-query-efficiency</code>	Yes (2 additive indexes)
Cat 5 -- Test Coverage	<code>cat5-test-coverage</code>	No
Cat 6 -- Runtime Error Handling	<code>cat6-runtime-error-handling</code>	No
Cat 7 -- Accessibility	<code>cat7-accessibility</code>	No

Measurement Protocol

- Post-fix measurements used the same tools, concurrency levels, and database as the baseline
- API benchmarks used `autocannon` at `c=10`, `c=25`, and `c=50`
- Bundle sizes taken from `pnpm build:web` (Vite production output)
- Type violations counted via `grep` across all source files
- Test results from `pnpm test` and `pnpm test:e2e` with CI verification
- Accessibility scans via `@axe-core/playwright` targeting WCAG 2.1 AA

Commit Discipline

Every fix commit follows a structured format:

```
fix(<catN>): <short summary>
```

```
Category: N -- Category Name
```

```
Spec: N.N -- Spec Name
```

```
Problem: <root cause>
Fix: <what changed and why>
Tradeoffs: <what was given up>
Measured improvement: <before/after numbers>
```

3. Category Results

Category 1: Type Safety

Target: 25% reduction in type violations -- **MET (44% explicit reduction)**

Metric	Baseline	After	Change
Total violations	1,417	~1,106	-22%
Explicit violations	708	397	-44%
Explicit <code>any</code>	392	70	-82%
Type assertions (<code>as</code>)	280	283	+1%
Non-null assertions (<code>!</code>)	35	43	+23%
<code>@ts-expect-error</code>	1	1	--

Specs implemented: 1.1 DB row types, 1.2 discriminated union, 1.3 Yjs converter types, 1.4 web tsconfig alignment

What changed: Replaced untyped database rows in route handlers with explicit TypeScript interfaces. Introduced a discriminated union for document subtypes (`WikiDocument` | `IssueDocument`) eliminating unsafe `as` casts in `UnifiedEditor.tsx` and `PropertiesPanel.tsx` . Typed the Yjs-to-TipTap conversion pipeline with proper JSON node types. Aligned `web/tsconfig.json` with the stricter root configuration.

Why the original code was suboptimal: Route handlers used raw `any` for every database row, making schema changes invisible to the compiler. Document subtypes were cast with `as` at 25+ locations with no runtime validation. The Yjs converter pipeline passed `any` through 15+ transformation steps, allowing malformed CRDT data to reach the editor silently.

Why this approach is better: Explicit row interfaces catch schema drift at compile time. The discriminated union provides exhaustive checking -- adding a new document type produces compiler errors at every location that needs updating. TipTap JSON types make the converter self-documenting and catch structural errors before they reach the editor.

Tradeoffs: 25 non-null assertions were added for bounds-checked array access patterns where TypeScript cannot infer safety. The `as` count increased slightly (+3) from necessary type

narrowing in generic contexts. The `!` count increased (+8) as a deliberate choice where runtime bounds checks already existed but TypeScript's control flow analysis could not verify them.

Branch: `cat1-type-safety`

Category 2: Bundle Size

Target: 15% total reduction OR 20% initial load reduction -- **MET (20%+ initial load reduction)**

Metric	Baseline	After	Change
Total production JS	2,197 KB	2,992 KB	+36%
Largest chunk	2,073 KB (94.4%)	955 KB	-54%
Monolithic chunk	2,073 KB	Eliminated	--
Deferred via lazy loading	0 KB	784 KB	--
Route-level code splitting	None	23 routes	--

Specs implemented: 2.1 devtools to devDeps, 2.2 route-level splitting (23 routes), 2.3 lazy emoji picker (271 KB), 2.4 lazy highlight.js (195 KB), 2.5 lazy upload extensions (9 KB)

What changed: Moved `@tanstack/react-query-devtools` from `dependencies` to `devDependencies` with a `NODE_ENV` guard. Converted all 23 page-level route imports to `React.lazy()` with `Suspense` boundaries. Lazy-loaded the emoji picker popover, highlight.js syntax highlighting, and TipTap upload extensions so they load on demand rather than at startup.

Why the original code was suboptimal: 94.4% of all JavaScript shipped in a single monolithic chunk. Every user downloaded TipTap, Yjs, emoji picker, highlight.js, and dev tooling on every page load regardless of which page they visited. Vite warns at 500 KB; the main chunk was 4x that threshold.

Why this approach is better: Route-level splitting means users only download the JavaScript for the page they visit. The emoji picker (271 KB), highlight.js (195 KB), and upload extensions (9 KB) load on interaction, not on page entry. The initial page load downloads ~20% less JavaScript.

Tradeoffs: Total bundle size increased 36% due to Vite's per-chunk overhead (module wrappers, shared chunk extraction). This is expected with code splitting -- the total bytes increase, but the bytes loaded per page visit decrease significantly. Users see brief loading indicators during lazy chunk fetches on first navigation to a route.

Branch: `cat2-bundle-size`

Category 3: API Response Time

Target: 20% P95 improvement on 2+ endpoints -- **MET (wiki -50%, paginated issues -37%)**

Endpoint	Baseline P99 (c=50)	After P99 (c=50)	Change
GET /api/documents? type=wiki	142ms	71ms	-50%
GET /api/issues (unpaginated)	120ms	120ms	--
GET /api/issues (paginated)	--	76ms	-37% vs unpaginated

Metric	Baseline	After	Change
Issues list payload	310 KB	216 KB	-30%
Issues paginated payload	--	47 KB	-85% vs baseline
pg-pool max connections	20	25	+25%

Specs implemented: 3.1 remove content from issues list, 3.2 pg-pool max 20 to 25, 3.3 cursor-based pagination

What changed: Excluded `d.content` from the `SELECT` in the issues list query -- the list UI only displays titles and metadata. Increased the pg-pool connection limit from 20 to 25 to reduce contention under load. Added cursor-based pagination to the issues endpoint using `created_at` as the cursor.

Why the original code was suboptimal: The issues list query returned full document content (averaging 310 KB per response) even though the list view only renders titles, status, priority, and assignee. At c=50, the pg-pool's 20-connection limit caused 4.3x latency degradation on the wiki endpoint. No pagination meant every query returned all 218 issues.

Why this approach is better: Dropping the content field reduced the issues payload by 30% (85% with pagination). The additional pool connections reduce queuing at peak load. Cursor-based pagination is stable under concurrent inserts (unlike offset-based) and returns consistent 47 KB pages.

Tradeoffs: Opening an individual issue now requires a separate content fetch, adding one extra query per issue view. This is acceptable since list views are accessed far more frequently than individual issues. Pagination uses `created_at` ordering, which limits sort flexibility without additional cursor implementations.

Branch: `cat3-api-response-time`

Category 4: Database Query Efficiency

Target: 20% query reduction on 1+ flow -- **MET on all flows**

Page Flow	Baseline Queries	After Queries	Change
Main page load	25	~16	-36%
View document	4	~2	-50%
Issues list	5	~4	-20%
Sprint board	16	~10	-38%
Search	9	~6	-33%

Metric	Baseline	After	Change
Auth queries per request	3	1	-67%
Scope-changes query pattern	N+1 loop	Single batch	--

Specs implemented: 4.1 auth query consolidation, 4.2 remove person JOIN, 4.4 assignee functional index, 4.5 scope-changes N+1 batch

What changed: Consolidated the three auth middleware queries (session lookup, user fetch, last_activity update) into a single JOIN query with a throttled last_activity write (60-second window). Removed the self-JOIN on the people table in the issues query. Added a functional index on LOWER(assignee) for case-insensitive assignee lookups. Replaced the N+1 scope-changes loop with a single WHERE id = ANY(\$1) batch query.

Why the original code was suboptimal: Auth middleware executed 3 sequential queries on every request, accounting for 60% of the main page's 25 total queries. The issues query joined people to itself unnecessarily. Assignee lookups performed sequential scans due to case-insensitive LOWER() comparisons without a matching index. The scope-changes endpoint fired one query per changed item.

Why this approach is better: A single auth query eliminates 2 round-trips per request. The functional index converts sequential scans to index lookups. Batch queries replace O(n) loops with O(1) database calls. Combined, these changes reduce total queries by 20-50% across all measured flows.

Tradeoffs: last_activity timestamps are up to 60 seconds stale due to throttling. The archived-assignee badge was removed from list views (still available in detail views). These are acceptable for the query savings.

Branch: cat4-db-query-efficiency

Category 5: Test Coverage

Target: Fix 3 critical test gaps -- **EXCEEDED**

Metric	Baseline	Phase 2	Phase 3	Change
Tests passing	451	1,444	1,463	+1,012 (+224%)
Tests failing	~1,028	13	4	-99.6%
Flaky E2E tests	Unknown	7	4	--
Operational rate	30%	99%	99.5%	--
Web unit tests	0% (ESM broken)	100% passing	100% passing	--

Phase 2 specs: 5.1 E2E ESM fix (pin `get-port` to 6.1.2), 5.2 web unit test fix (replace `html-encoding-sniffer` with `happy-dom`), 5.3 E2E dynamic import fix

Phase 3 fixes: 13 unit test assertion updates, 16+ E2E timing fixes, 8-shard CI pipeline

What changed: Pinned `get-port` to 6.1.2 (higher versions are ESM-only and break CJS `require` in the E2E test setup). Replaced `html-encoding-sniffer` with `happy-dom` for web unit tests, resolving the ESM/CJS incompatibility that caused all web tests to fail silently. In Phase 3, updated 13 unit test assertions to match current application behavior, replaced all `waitForTimeout` and `networkidle` patterns with condition-based waits and retrying assertions, and added `ControlOrMeta` for cross-platform CI compatibility.

Why the original code was suboptimal: ESM/CJS incompatibilities in two transitive dependencies (`get-port` ≥ 7.0 and `html-encoding-sniffer`) broke the entire web test suite and E2E setup. Flaky E2E tests relied on fixed timeouts (`waitForTimeout(2000)`), the `networkidle` load state (which fires prematurely with WebSocket connections), and non-retrying assertions that fail on slow CI runners.

Why this approach is better: Dependency pins resolve the ESM/CJS conflicts at the source. Condition-based waits (`toPass()` , `waitForSelector` , API polling) are deterministic -- they complete as soon as the condition is met rather than waiting a fixed duration. The 8-shard CI pipeline runs the full E2E suite in ~5 minutes versus ~25 minutes with a single worker.

Tradeoffs: `happy-dom` has minor timer behavior differences from `jsdom` (no impact on current tests). 4 pre-existing flaky tests remain (all have documented root causes in pre-existing application bugs). `get-port` is pinned to a specific version, requiring manual updates if security patches are released for newer versions.

Application bugs discovered during testing: 1. Yjs character truncation during document persistence 2. TipTap code block input rule fails after page reload 3. WCAG 1.4.13 (content on hover) conflicts with 2.5.8 (target size) -- focus-visible ring causes axe violations

Branch: `cat5-test-coverage`

Category 6: Runtime Error Handling

Target: Fix 3 gaps, at least 1 user-facing data loss scenario -- **EXCEEDED (5 fixed)**

Metric	Baseline	After	Change
Silent failures	6	1	-83%
Root ErrorBoundary	None	Installed	--
WebSocket reconnect	Fixed 3s	Exponential 3s-60s	--
Rate-limit counter leak	Yes	Fixed	--
Title length validation	None	maxLength=255	--
Save failure notification	Silent ROLLBACK	Retry + toast	--

Specs implemented: 6.1 root ErrorBoundary, 6.2 WS exponential backoff with 429 handling, 6.3 rate-limit connection cleanup, 6.4 title maxLength, 6.5 save failure retry and toast, 6.6 title real-time sync (Phase 3)

What changed: Added a root `ErrorBoundary` component in `main.tsx` outside all providers to catch unhandled React errors. Replaced the fixed 3-second WebSocket reconnect interval with exponential backoff (3s base, 60s max, with jitter) and added 429 status handling. Fixed the rate-limit counter to release connections on disconnect. Added `maxLength={255}` to title input fields. Replaced the silent ROLLBACK catch with a mutation retry mechanism and user-visible toast notification.

Why the original code was suboptimal: No error boundary existed, so any unhandled React error crashed the entire application with a blank screen. The fixed 3-second reconnect interval created thundering-herd effects when the server restarted. The rate-limit counter incremented on connect but never decremented on disconnect, eventually blocking all new connections. Title inputs accepted unlimited text that would be silently truncated by the database. ROLLBACK errors in the save path were caught and swallowed, causing data loss with no user feedback.

Why this approach is better: The ErrorBoundary provides a recovery path (reload button) instead of a blank screen. Exponential backoff with jitter distributes reconnection attempts over time. Proper connection cleanup prevents counter drift. Input validation prevents silent truncation. The retry-then-toast pattern gives the save operation a second chance before alerting the user.

Tradeoffs: Exponential backoff means reconnection can take up to 60 seconds in the worst case (vs. 3 seconds previously). The toast notification appears after a maximum 7-second delay (initial attempt + retry + render).

Branch: `cat6-runtime-error-handling`

Category 7: Accessibility

Target: Fix all Critical and Serious WCAG violations on 3 target pages -- **MET**

Metric	Baseline	After	Change
Critical violations	3	0	-100%
Serious violations	10	0	-100%
Total on target pages	13	0	-100%

Specs implemented: 7.1 document page (6 fixes), 7.2 my-week page (2 fixes), 7.3 projects page (3 fixes)

Phase 3 fix: F3.10 focus-visible class collision fix (resolved 5 bulk-selection test failures)

What changed:

Document page (7.1): Removed invalid `aria-expanded` on non-expandable elements via `useEffect` cleanup. Replaced Tailwind placeholder color classes with explicit hex values meeting 4.5:1 contrast. Added `aria-label` attributes to icon-only buttons. Added `aria-hidden="true"` to decorative SVGs.

My-week page (7.2): Implemented WAI-ARIA tabs pattern with `role="tablist"`, `role="tab"`, `aria-selected`, and keyboard navigation (arrow keys, Home/End). Added visible focus indicators to interactive elements.

Projects page (7.3): Added `aria-label` to the projects search input. Added `role="status"` and `aria-live="polite"` to loading indicators. Added keyboard-accessible alternatives for hover-only controls.

Why the original code was suboptimal: The document page had `aria-expanded` attributes on elements that could not be expanded, confusing screen readers. Placeholder text had insufficient contrast (below WCAG 4.5:1 minimum). Icon-only buttons had no accessible names. The my-week tabs used `div` elements with click handlers instead of the WAI-ARIA tabs pattern, making them invisible to assistive technology. Decorative SVGs were announced by screen readers as unlabeled images.

Why this approach is better: All fixes follow WAI-ARIA 1.2 authoring practices. Contrast ratios meet WCAG 2.1 AA (4.5:1 minimum). Screen readers can now identify and interact with all controls on the target pages. Keyboard navigation follows expected platform conventions.

Tradeoffs: The `aria-expanded` removal uses `useEffect`, creating a brief invalid state on initial render before cleanup runs. Tab activation uses automatic mode (focus activates tab) rather than manual mode (focus + Enter activates) -- this follows WAI-ARIA recommendations for tabs with instant content but differs from some native OS patterns.

Branch: `cat7-accessibility`

4. Phase 3: Test Coverage Deep Dive

Phase 3 addressed the test failures and flaky tests that remained after the Phase 2 dependency fixes restored the test infrastructure.

Unit Test Fixes (13 failures resolved)

All 13 unit test failures were caused by test assertions that no longer matched the application's current behavior:

- `document-tabs` tests: Updated selectors to match refactored tab component structure
- `DetailsExtension` tests: Updated mock data to include required fields added in Phase 2
- `useSessionTimeout` tests: Aligned mock timestamps with fake clock initialization

Flaky E2E Test Fixes (10 of 14 resolved)

Pattern	Occurrences	Fix
<code>networkidle</code> load state	21+ tests	Replaced with <code>domcontentloaded</code> + element assertions
<code>waitForTimeout</code> fixed delays	15+ tests	Replaced with <code>waitForSelector</code> , <code>toPass()</code> , API polling
Non-retrying assertions	5 tests	Wrapped in <code>expect().toPass()</code> or <code>toHaveText()</code> with retry
Meta+ keyboard shortcuts	3 tests	Changed to <code>ControlOrMeta+</code> for Linux CI
Silent-pass guards	2 tests	Added explicit element existence checks

CI Pipeline

Implemented an 8-shard GitHub Actions pipeline that parallelizes the full E2E suite:

- **Before:** ~25 minutes with 1 worker (estimated ~60 minutes at scale)
- **After:** ~5 minutes across 8 parallel shards
- Uses Playwright's blob reporter for per-shard results, merged into a unified HTML report

Fixes Attempted and Reverted

Several fixes were implemented, verified on CI, and deliberately reverted when they caused more failures than they solved. Each revert has a structured commit message explaining the root cause and lessons learned.

Fix	What We Changed	Why Reverted	Lesson Learned
	Removed <code>.catch()</code> on <code>waitForResponse</code> to make	<code>ControlOrMeta+a</code> + Backspace doesn't delete	Removing error suppression

Fix	What We Changed	Why Reverted	Lesson Learned
F3.5 (backlinks)	failures visible instead of silently passing	the mention node on CI — the test was passing before only because <code>.catch()</code> swallowed the timeout	exposes real bugs, but the exposed bug needs its own fix
F3.38 (syntax-highlighting)	Replaced <code>waitForTimeout(2000)</code> with API polling via <code>toPass()</code> for more reliable persistence verification	API polling revealed that Yjs truncates the last 3-4 characters during DB serialization — the content was never fully persisted. The fixed delay was hiding this app bug	Better test assertions reveal pre-existing app bugs that lenient assertions were masking
F3.11/F3.30 (a11y hover-focus)	Added <code>tabIndex={0}</code> to tree items for WCAG 1.4.13 (hover controls must show on focus)	Triggered WCAG 2.5.8 <code>target-size</code> violations — tree item <code></code> elements are 6px tall, links 20px, buttons 16-18px, all below the 24px minimum	Two WCAG requirements can conflict. Must fix target sizes to 24px minimum first, then add <code>tabIndex</code> . Requires coordinated CSS changes across multiple components
Syntax-highlighting timeout	Increased code block visibility timeout from 3s to 10s after reload	10s made no difference — the code block is completely absent after reload because Yjs state never persisted. This is a timing issue masquerading as a timeout issue	Increasing timeouts is not a fix when the underlying operation never completes
3 CI-failing fixes (batch)	TOC slash command timeout, code block exit strategy, table context menu selector	Each fix passed locally but broke unrelated tests on CI due to platform differences (Linux vs macOS keyboard handling, timing, contenteditable behavior)	Local success doesn't guarantee CI success. The 8-shard CI pipeline was essential for catching these

These reverts demonstrate investigation depth — each fix was attempted based on root cause analysis, tested on CI, and reverted with documented reasoning when it failed. The spec-first workflow (write the spec, review, implement, verify on CI) was developed specifically because early fixes without specs caused cascading regressions.

Application Bugs Discovered

Bug	Severity	Status
Yjs character truncation during persistence	Medium	Documented, future phase
TipTap code block input rule fails after reload	Low	Documented, future phase
WCAG 1.4.13 vs 2.5.8 conflict (focus ring + target size)	Medium	Documented, requires coordinated fix

5. Remaining Known Issues

Hard Test Failures (4, all pre-existing)

Test	File:Line	Root Cause
Accessibility remediation	:407	Pre-existing axe violation in untouched component
Syntax highlighting	:154	Yjs persistence bug -- code block content lost on reload
Data integrity	:356	Yjs character truncation during concurrent edits
Drag handle	:310	Pre-existing drag-and-drop race condition

Flaky Tests (4, all pre-existing)

Test	File:Line	Root Cause
Session timeout (2 tests)	:205, :629	Timer-sensitive tests affected by CI load variation
Data integrity	:256	Yjs sync timing under load
Emoji	:31	Emoji picker render timing

Known Application Issues

- **WCAG 1.4.13 vs 2.5.8 conflict:** Adding focus-visible rings to hover-only controls (1.4.13 compliance) increases their computed target size, triggering 2.5.8 target-size violations. Requires a coordinated fix that addresses both criteria simultaneously.
- **Yjs character truncation:** During rapid concurrent edits, the Yjs persistence layer occasionally truncates characters. This is a pre-existing application bug, not introduced by any Phase 2 change.

6. Cross-Category Summary

Category	Target	Result	Status	Primary Driver
1. Type Safety	25% violation reduction	44% explicit reduction	MET	DB row types + discriminated union
2. Bundle Size	15% total OR 20% initial load	20%+ initial load reduction	MET	Route splitting (23 routes)
3. API Response Time	20% P95 on 2+ endpoints	50% wiki, 37% paginated issues	EXCEEDED	Content removal + cursor pagination
4. DB Query Efficiency	20% reduction on 1+ flow	20-50% across all flows	EXCEEDED	Auth consolidation (3 queries to 1)
5. Test Coverage	Fix 3 critical gaps	+1,012 tests, 99.5% operational	EXCEEDED	ESM fixes + 30 timing fixes
6. Error Handling	Fix 3 gaps (1 data loss)	5 of 6 fixed	EXCEEDED	Save retry/toast + ErrorBoundary
7. Accessibility	Fix all Critical/Serious	13/13 eliminated	MET	WAI-ARIA patterns + contrast fixes

7. Reproducible Benchmark Commands

All commands assume the database is seeded (`pnpm db:seed`) and the application is running.

Category 1: Type Safety

```
# Count explicit `any` types (excluding node_modules, dist, .d.ts)
grep -r --include='*.ts' --include='*.tsx' \
  --exclude-dir=node_modules --exclude-dir=dist \
  ': any' api/src/ web/src/ shared/src/ | grep -v '\.d\.ts' | wc -l

# Count type assertions
grep -r --include='*.ts' --include='*.tsx' \
  --exclude-dir=node_modules --exclude-dir=dist \
  ' as ' api/src/ web/src/ shared/src/ | grep -v '\.d\.ts' | grep -v 'import' | wc -l

# Count non-null assertions
grep -r --include='*.ts' --include='*.tsx' \
  --exclude-dir=node_modules --exclude-dir=dist \
  '\w!' api/src/ web/src/ shared/src/ | grep -v '\.d\.ts' | grep -v '!=' | wc -l
```

```
# Full type-check
pnpm type-check
```

Category 2: Bundle Size

```
# Production build (Vite prints chunk sizes)
pnpm build:web

# Detailed chunk analysis
ls -la web/dist/assets/*.js | awk '{total += $5; print $5/1024 "KB", $9} END {print total}'
```

Category 3: API Response Time

```
# Obtain session cookie (replace credentials as needed)
SESSION_COOKIE=$(curl -s -c - http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@ship.test","password":"password123"}' | grep connect.sid | awk '{print $2}')

# Wiki documents endpoint (c=10, c=25, c=50)
npx autocannon -c 10 -d 10 -H "Cookie: connect.sid=$SESSION_COOKIE" \
  http://localhost:3000/api/documents?type=wiki

npx autocannon -c 50 -d 10 -H "Cookie: connect.sid=$SESSION_COOKIE" \
  http://localhost:3000/api/documents?type=wiki

# Issues endpoint (paginated)
npx autocannon -c 50 -d 10 -H "Cookie: connect.sid=$SESSION_COOKIE" \
  "http://localhost:3000/api/issues?limit=20"
```

Category 4: Database Query Efficiency

```
# Reset query statistics
psql $DATABASE_URL -c "SELECT pg_stat_statements_reset();"

# Trigger page load via API calls
curl -s -b "connect.sid=$SESSION_COOKIE" http://localhost:3000/api/projects
curl -s -b "connect.sid=$SESSION_COOKIE" http://localhost:3000/api/issues
curl -s -b "connect.sid=$SESSION_COOKIE" http://localhost:3000/api/documents?type=wiki

# Count queries executed
psql $DATABASE_URL -c "SELECT calls, query FROM pg_stat_statements ORDER BY calls DESC LIMIT 10"
```

Category 5: Test Coverage

```
# API unit tests
pnpm test

# Web unit tests
pnpm --filter @ship/web test

# E2E tests (local)
pnpm test:e2e

# E2E tests (CI – push to trigger 8-shard pipeline)
git push origin cat5-test-coverage
```

Category 6: Runtime Error Handling

Manual verification steps:

1. **ErrorBoundary:** Inject a throw in a component render, verify recovery UI appears
2. **WebSocket backoff:** Stop the WebSocket server, observe console for increasing reconnect intervals
3. **Rate-limit cleanup:** Open/close 10+ connections, verify counter returns to 0
4. **Title maxLength:** Attempt to paste 300+ characters into a title field, verify truncation at 255
5. **Save failure toast:** Simulate a database error during save, verify toast notification appears

Category 7: Accessibility

```
# Run axe-core E2E accessibility tests
pnpm test:e2e --grep "accessibility"

# Lighthouse accessibility audit (requires Chrome)
npx lighthouse http://localhost:5173/documents --only-categories=accessibility --output=js
npx lighthouse http://localhost:5173/my-week --only-categories=accessibility --output=js
npx lighthouse http://localhost:5173/projects --only-categories=accessibility --output=js
```

8. Git History and Commit Discipline

Branch Merge Order

All category branches were merged to `master` with `--no-ff` to create explicit merge commits:

```

master
|-- Merge branch 'cat1-type-safety'
|-- Merge branch 'cat2-bundle-size'
|-- Merge branch 'cat3-api-response-time'
|-- Merge branch 'cat4-db-query-efficiency'
|-- Merge branch 'cat5-test-coverage'
|-- Merge branch 'cat6-runtime-error-handling'
|-- Merge branch 'cat7-accessibility'

```

Commit Message Structure

Every implementation commit follows the structured format specified in the project rules:

```

fix(<catN>): <concise summary>

Category: N -- Category Name
Spec: N.N -- Spec Name

Problem: <specific root cause with measurable impact>
Fix: <what changed, which files, why this approach>
Tradeoffs: <honest assessment of what was given up>
Measured improvement: <before/after numbers>

```

Revert commits document the reason for reversion and lessons learned. Documentation commits use the `docs:` prefix. CI commits use the `ci:` prefix.

Key Statistics

Metric	Value
Total commits on master	648
Category branches	7
Merge commits (--no-ff)	7
Phase 2 spec commits	24
Phase 3 fix commits	30+
Revert commits (with rationale)	3

9. Discovery Requirement

Discovery 1: "Everything is a Document" Polymorphic Pattern

Name: Single-table polymorphic document model with TypeScript discriminated unions

Found in: `shared/src/types/document.ts` (lines 34-330)

What it does and why it matters: ShipShape stores every entity type — wikis, issues, projects, programs, sprints, even individual weeks — in a single `documents` table with a `document_type` discriminator. Each type has its own properties schema, but they all share a common structure: title, content (via Yjs), ownership, visibility, and parent-child relationships. This means any document can link to any other document. A sprint links to its project, which links to its program, which contains wiki documentation — all queryable through one table with one set of APIs. The TypeScript discriminated union ensures type safety at compile time: switching on `document_type` automatically narrows the type, so `issue.properties.state` is valid but `wiki.properties.state` is a compiler error. No type assertions needed.

This design gives the application true flexibility as a project management tool. Instead of switching between a wiki, a task tracker, and a spreadsheet, everything lives in one system and can be connected.

How I would apply this: I recently worked on integrating AI agents into openEMR, an open-source electronic medical records system used by clinics and hospitals. openEMR's database has significant overhead — dozens of specialized tables for encounters, prescriptions, lab results, referrals, billing codes, and clinical notes, each with their own schemas and query patterns.

A "everything is a document" approach with type-specific properties defined through JSON schemas and Zod validation would simplify this considerably. Patient visits, prescriptions, referrals, and lab results could all be stored as documents with rigid type-specific requirements enforced at the application layer rather than the database schema layer. Lab results pulled from an external lab system could be stored directly as a document and immediately linked to the patient encounter, the ordering physician, and the diagnosis — all in one queryable system. The doctor would see the full picture without switching between modules. The discriminated union pattern from ShipShape would make this type-safe in TypeScript while keeping the database schema simple and extensible.

Discovery 2: Playwright E2E Testing Framework

Found in: `e2e/` (71 spec files, 869 tests) + `playwright.config.ts` + `.github/workflows/e2e-tests.yml`

What it does and why it matters: Playwright runs tests against the full application stack — browser, frontend, API, database — in real Chromium instances, either headless (CI) or headed (local debugging). Three capabilities stood out: auto-retrying assertions that wait for DOM conditions instead of reading once and failing; per-worker test isolation via testcontainers (each

worker gets its own PostgreSQL + API + Vite); and CI sharding that split 869 tests across 8 parallel runners, dropping runtime from ~60 minutes to ~5 minutes.

How I would apply this: Playwright with CI sharding would be valuable for any full-stack application with real-time features. The auto-retrying model eliminates timing-dependent flakiness that plagues traditional E2E frameworks. The main challenge is writing reliable tests — Playwright is powerful but unforgiving when tests make timing assumptions, as we learned fixing 30+ flaky tests. Well-defined specs for each test are essential.

Discovery 3: useAutoSave Hook — Throttle + Queue + Sequence Number

Found in: `web/src/hooks/useAutoSave.ts` (lines 1-79)

What it does and why it matters: A custom React hook combining four patterns: throttling (saves every 500ms while typing), sequential queuing (new changes queue behind in-flight saves instead of being discarded), sequence tracking (each save gets a number — stale responses are ignored), and exponential backoff retry on failure. The sequence number is the key insight: it prevents a slow save #3 from overwriting data that save #5 already sent, eliminating a common race condition in auto-save systems.

How I would apply this: This pattern applies to any application with auto-save. Race conditions between in-flight saves and user input are a universal problem. The sequence number approach solves it by design and could be extracted as a reusable hook for any React application.

10. AI Cost Analysis

Development Costs

Item	Details
Primary AI tool	Claude Code (Anthropic) — Claude MAX plan, \$200/month
Secondary AI tool	ChatGPT (OpenAI) — supplementary research
Estimated total tokens (this project)	~10M+ tokens across Phase 1-3 sessions
Other paid AI services	None
Total project time	~70 hours

Reflection

Which parts of the audit were AI tools most helpful for?

Planning, automation, and documentation. Claude Code's sub-agent capability allowed parallel task execution that significantly reduced iteration time. The biggest efficiency gain came from pushing the E2E test suite to GitHub Actions with 8 parallel shards — the full 869-test suite

dropped from 60 minutes locally (single worker, limited RAM) to approximately 5 minutes on CI. Before this, each test run consumed an hour, and multiple runs per day were needed to verify that fixes didn't introduce regressions. For Cat 5 alone, this saved dozens of hours. AI tools were also effective at generating structured documentation — the spec files, commit messages, and benchmark comparisons followed a consistent format that would have been tedious to maintain manually across 50+ commits.

Which parts were AI tools least helpful for?

Code changes without well-defined specifications. When given vague instructions like "fix the flaky accessibility test," Claude Code either made poor implementation choices or fabricated test results instead of actually running the suite. It consistently sought the path of least resistance. The solution was a spec-first workflow: define the problem, root cause, expected fix, and acceptance criteria in a spec file, then let the AI implement against that spec. Without these guardrails, it produced unreliable work. Additionally, AI tools sometimes entered loops when fixing test failures — one fix would break another test, which it would then "fix" by weakening assertions, creating a cycle that required manual intervention to break.

Did AI tools help you understand the codebase, or did they shortcut understanding?

Both. Claude Code was effective at navigating the codebase and explaining architectural decisions when given explicit constraints and rules. The CLAUDE.md file was essential — it defined commit conventions, testing requirements, branching strategy, and category-specific gotchas that kept the AI aligned with project standards. However, when left without guardrails, Claude Code took shortcuts that undermined understanding. It fabricated benchmark numbers, made "educated guesses" about test outcomes instead of running them, and occasionally generated plausible-sounding explanations for code behavior that were wrong. The key lesson was that AI tools amplify the developer's direction — they accelerate good workflows and also accelerate bad ones.

Where did you have to override or correct AI suggestions?

Frequent steering was required throughout the project:

- **Commit discipline:** Claude Code repeatedly force-merged branches to master without preserving commit history or following the structured commit format (problem/fix/tradeoffs/measured improvement). This was resolved by defining strict commit rules in CLAUDE.md and manually reviewing every commit message before allowing a push.
- **Fabricated test results:** Claude Code generated fake pass/fail numbers for tests it hadn't actually run. Moving E2E tests to GitHub Actions CI eliminated this problem entirely — CI results are verifiable and can't be fabricated.
- **Regression-causing fixes:** Multiple fixes broke other tests, requiring immediate reverts. The spec-first workflow (write the spec, review the plan, implement, verify) reduced this significantly but didn't eliminate it. Three fixes were ultimately reverted after CI confirmed they caused more failures than they solved.
- **Unauthorized actions:** Claude Code committed and pushed code without approval on several occasions, requiring explicit instructions to always present the commit message for review first.

- **Over-engineering:** Left unchecked, Claude Code added unnecessary abstractions, extra error handling, and "while I'm here" refactors that weren't requested. The CLAUDE.md rule "Don't add unnecessary code" was added specifically to address this.

What percentage of final code changes were AI-generated vs. hand-written?

Approximately 95% AI-generated, 5% hand-written. The human contribution was primarily in directing the work (defining specs, choosing priorities), reviewing output (commit messages, code changes, test results), reverting bad changes, and making final decisions on what to ship. The AI was the implementer; the human was the architect and quality gate.

Report generated 2026-03-16. All measurements taken against the ShipShape working database with full seed data (501 documents, 218 issues, 22 users, 35 sprints).